

SQL et Git

Où une appli range ses données, SQL pour interroger une base (SELECT, WHERE, GROUP BY, un premier JOIN), puis Git et GitHub pour mettre son premier projet en ligne.

ON REPART AVEC · Le premier projet en ligne sur GitHub, avec un README qui l'explique.

01 OÙ VIVENT LES DONNÉES D'UNE APPLI 02 UNE REQUÊTE SE LIT COMME UNE PHRASE
03 RELIER DEUX TABLES 04 GIT : DES POINTS DE SAUVEGARDE 05 ENTRE DEUX SÉANCES

À LA FIN DE LA SÉANCE

Tu sauras dire où une appli range ses données et lire une requête SQL comme une phrase.
Tu sauras filtrer, trier, compter, regrouper et relier deux tables, en SQL comme en pandas.
Tu sauras ce que font add, commit, push et pull, et pousser un notebook sur GitHub depuis Colab.

Où vivent les données d'une appli

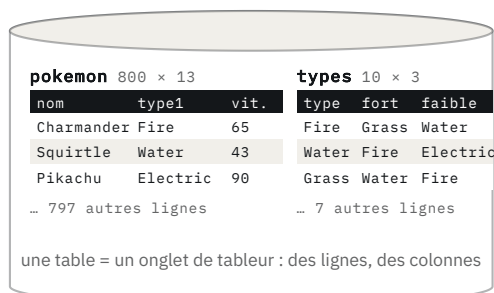
Derrière Instagram, Doctolib ou ton jeu, pas de fichier CSV : une base de données, faite de tables, et une langue pour l'interroger.

1. l'appli, ce que tu vois



une liste, un score, un profil...

2. la base de données, ce que tu ne vois pas



3. SQL, la langue pour lui parler

```
SELECT nom FROM pokemon
WHERE type1 = 'Fire'
```

52 lignes, les Pokémon Feu

une question, un tableau en réponse

Une appli garde ses données dans des tables ; SQL est la langue pour leur poser des questions.

- SQL A CINQUANTE ANS, SQLITE EST DANS TA POCHE

SQL est né chez IBM en 1974, sous le nom SEQUEL, et tout le monde le parle encore : les sites, les banques, les trains. Il ne fait pas tourner l'ordinateur, il pose des questions à des tables ; c'est pour ça qu'il a survécu à tous les langages à la mode.

SQLite est une base de données complète qui tient dans un seul fichier, sans serveur. Chaque téléphone en contient : les contacts, les messages, l'historique du navigateur y sont rangés. En Python, le module `sqlite3` est fourni d'office : le notebook crée sa base en mémoire, sans rien installer.

Une requête se lit comme une phrase

la question, en SQL

```
SELECT nom, vitesse
FROM pokemon
WHERE type1 = 'Fire'
ORDER BY vitesse DESC
LIMIT 3
```

une seule phrase : « les 3 Pokémon Feu les plus rapides »

ce que dit chaque ligne

- quelles colonnes
- quelle table
- quelle condition sur les lignes
- dans quel ordre
- combien de lignes

la réponse, un tableau

nom	vitesse
Talonflame	126
Infernape	108
Pyroar	106

3 lignes gardées sur les 52 Pokémon Feu,
2 colonnes gardées sur 13

sans WHERE, le tableau ferait 800 lignes ;
sans LIMIT, 52.

Une requête se lit comme une phrase : quoi, d'où, à quelle condition, dans quel ordre, combien.

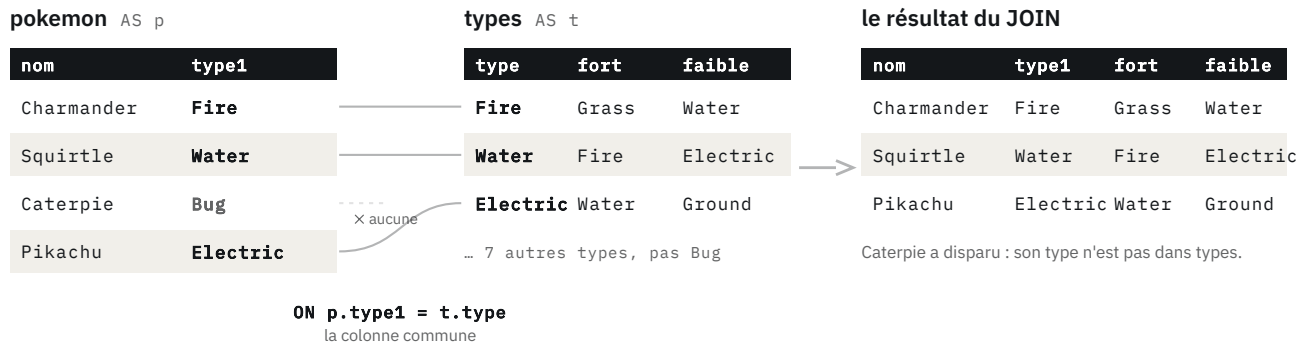
```
sql("SELECT nom, vitesse FROM pokemon WHERE type1 = 'Fire' ORDER BY vitesse DESC
LIMIT 3")
```

Les verbes de la séance 2 ont chacun leur traduction. Le tableau sert de fiche à retrouver ensuite.

LE VERBE	EN SQL	EN PANDAS	EXEMPLE
Filtrer	WHERE type1 = 'Fire'	df[df["type1"] == "Fire"]	52 Pokémon Feu
Trier	ORDER BY vitesse DESC LIMIT 5	df.sort_values("vitesse", ascending=False).head(5)	DeoxysSpeed Forme, 180
Compter	SELECT COUNT(*) ... WHERE legendaire = 1	len(df[df["legendaire"] == 1])	65 légendaires
Regrouper	SELECT type1, AVG(total) ... GROUP BY type1	df.groupby("type1") ["total"].mean()	Dragon, 550,5
Relier	JOIN types AS t ON p.type1 = t.type	df.merge(types, left_on="type1", right_on="type")	466 lignes recollées

Relier deux tables

Une vraie base éparpille l'information : les Pokémon d'un côté, les forces et faiblesses de chaque type de l'autre.



Sur les 800 Pokémon du notebook, **466** trouvent leur type dans la table types ; **334** restent dehors. `LEFT JOIN` les garde, avec des cases vides.

`JOIN` recolle deux tables par leur colonne commune ; ce qui n'a pas de correspondance disparaît.

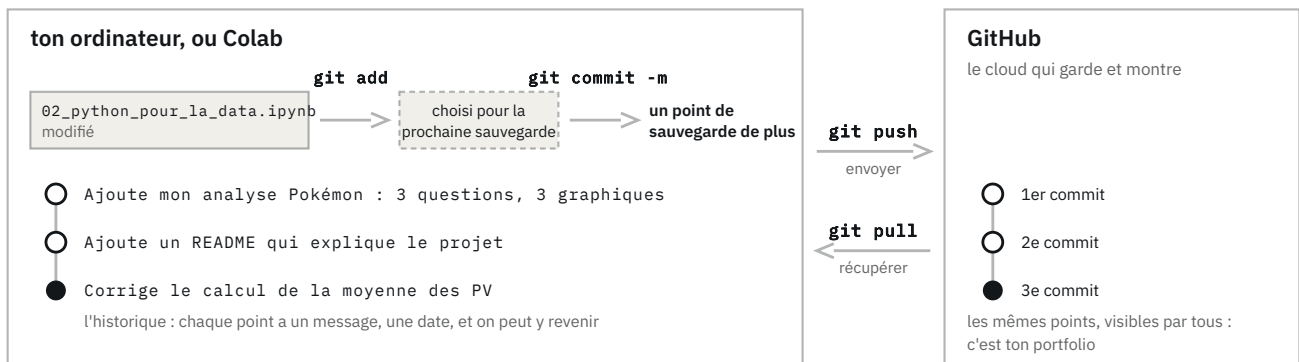
- LES OUBLIÉS DU JOIN

Huit types du dataset n'ont pas de ligne dans la table `types` : Bug, Dark, Dragon, Fairy, Ghost, Normal, Poison, Steel. Leurs 334 Pokémon disparaissent d'un `JOIN` simple, sans message d'erreur. C'est le piège classique : un résultat plus petit que prévu, et personne ne s'en aperçoit.

`LEFT JOIN` garde toutes les lignes de la table de gauche ; celles qui n'ont pas de correspondance ont des cases vides (`NULL`). Pour compter les oubliés : `WHERE t.type IS NULL` .

Git : des points de sauvegarde

Tu connais les sauvegardes d'un jeu vidéo : on y revient quand on rate. Git fait ça pour ton code ; GitHub garde les sauvegardes en ligne et les montre.



Un bon message dit pourquoi : « Ajoute mon analyse Pokémon : 3 questions, 3 graphiques »
pas « modif », « test », « fichier final v2 » : dans six mois, personne ne saura ce qu'ils contiennent.

Git garde des points de sauvegarde ; GitHub les met en ligne et les montre.

QUELLE COMMANDE ? • SUR PAPIER

Pour chaque situation, choisis la commande Git qui convient.

`git add` : choisir les fichiers qui entreront dans la prochaine sauvegarde

`git commit -m` : créer le point de sauvegarde, avec un message qui dit pourquoi

`git push` : envoyer ses sauvegardes sur GitHub

`git pull` : récupérer ce qui a changé sur GitHub

1. Tu veux choisir quels fichiers modifiés entreront dans la prochaine sauvegarde.
2. Tu veux créer le point de sauvegarde, avec un message qui dit ce que tu as fait.
3. Tes sauvegardes sont sur ton ordinateur ; tu veux qu'elles apparaissent sur GitHub.
4. Tu ouvres ton projet sur un autre ordinateur et tu veux la dernière version qui est sur GitHub.
5. Tu as trois fichiers modifiés, mais tu ne veux en sauvegarder qu'un seul.
6. Dans Colab, « Enregistrer une copie dans GitHub » revient à trois commandes d'un clic. Laquelle n'en fait pas partie ?

Les réponses sont dans le corrigé, en fin de leçon.

À toi : ton projet en ligne

Depuis Colab, le menu Fichier, puis « Enregistrer une copie dans GitHub », revient à `add`, `commit` et `push` d'un clic.

Pousse ton notebook de la séance 2 dans ton dépôt `mon-portfolio-ia`, chemin `seance-02/02_python_pour_la_data.ipynb`, avec un message qui dit ce qu'il contient.

Écris le README : la cellule de la section 6 le génère depuis ta fiche, tu le colles sur GitHub. Modifie-le une deuxième fois : ton historique a deux versions.

Même question, deux langues (section 7) : trois défis à résoudre en SQL puis en pandas, compare dit si les deux réponses concordent. Le premier est résolu, les deux autres sont à toi.

Entre deux séances

Le carnet `03_exercices.ipynb` : 12 exercices, de 🌟 à 🌟🌟🌟, avec vérification automatique, indices et solutions repliées, dont deux sur Git et deux où SQL et pandas doivent tomber d'accord. Il s'ouvre par un quiz de trois questions, celles qu'on se pose à voix haute au début de la séance suivante.

Mini-quiz

MINI-QUIZ · SUR PAPIER

Une seule bonne réponse par question.

1. Tu veux les 3 Pokémon Feu les plus rapides. Quelle partie de la requête garde seulement les Pokémon Feu ?
 - A. SELECT nom, vitesse
 - B. WHERE type1 = 'Fire'
 - C. ORDER BY vitesse DESC
 - D. LIMIT 3
2. En pandas tu écrirais `df.groupby("type1")["total"].mean()`. En SQL ?
 - A. SELECT type1, total FROM pokemon ORDER BY total
 - B. SELECT AVG(total) FROM pokemon WHERE type1
 - C. SELECT type1, AVG(total) FROM pokemon GROUP BY type1
 - D. SELECT COUNT(*) FROM pokemon GROUP BY total
3. Un JOIN entre pokemon (800 lignes) et types (10 types) renvoie 466 lignes. Où sont passés les 334 autres Pokémon ?
 - A. SQL les a perdus par erreur
 - B. Leur type n'est pas dans la table types : un JOIN simple les laisse tomber
 - C. Ce sont les Pokémon légendaires
 - D. Ils étaient en double
4. Dans quel ordre tapes-tu ces trois commandes pour envoyer ton notebook sur GitHub ?
 - A. commit, add, push
 - B. add, commit, push
 - C. push, add, commit
 - D. add, push, commit

Les réponses sont dans le corrigé, en fin de leçon.

À retenir

1. Une appli garde ses données dans une base faite de tables ; SQL est la langue pour les interroger.
2. Une requête se lit comme une phrase, SELECT quoi, FROM où, WHERE à quelle condition, ORDER BY dans quel ordre, LIMIT combien.
3. SQL et pandas font la même chose, WHERE et filtre, ORDER BY et sort_values, GROUP BY et groupby, JOIN et merge.
4. Un JOIN recolle deux tables par une colonne commune, et laisse tomber ce qui n'a pas de correspondance.
5. Git garde des points de sauvegarde, GitHub les met en ligne ; add choisit, commit sauvegarde avec un message qui dit pourquoi, push envoie, pull récupère.

Corrigé

Quelle commande ?

1. **git add** · add ne sauvegarde rien encore : il met de côté ce qui fera partie du prochain point de sauvegarde. Le commit vient après.
2. **git commit -m** · Le commit enregistre une version. Son message est lu par celui qui relira l'historique : « Corrige le calcul de la moyenne des PV » dit tout, « modif » ne dit rien.
3. **git push** · Un commit reste sur la machine où il a été fait. push copie tes commits sur GitHub ; sans lui, ton dépôt en ligne ne bouge pas.
4. **git pull** · pull rapatrie ce qui a changé sur GitHub, dans l'autre sens que push. Dans Colab, ouvrir un notebook depuis l'onglet GitHub revient à un pull.
5. **git add** · C'est exactement le rôle de add : choisir. Tu ajoutes le seul fichier voulu, puis le commit ne contient que lui.
6. **git pull** · Le clic revient à add, commit et push : Colab prend le notebook, écrit le point de sauvegarde avec ton message directement dans ton dépôt GitHub. pull, c'est l'inverse : récupérer.

Mini-quiz

1. **B. WHERE type1 = 'Fire'** · SELECT choisit les colonnes, WHERE les lignes, ORDER BY l'ordre, LIMIT le nombre. Sans WHERE, la requête renverrait les trois plus rapides de tous les types, DeoxysSpeed Forme en tête avec 180.
2. **C. SELECT type1, AVG(total) FROM pokemon GROUP BY type1** · GROUP BY fait les tas, un par type ; AVG calcule la moyenne dans chaque tas. Les deux langues donnent le même classement, Dragon en tête avec 550,5.
3. **B. Leur type n'est pas dans la table types : un JOIN simple les laisse tomber** · Bug, Normal, Poison et cinq autres types n'ont pas de ligne dans types, donc pas de correspondance. Un JOIN simple ne garde que ce qui se recolle ; LEFT JOIN garderait les 800, avec des cases vides.
4. **B. add, commit, push** · add choisit les fichiers, commit crée le point de sauvegarde avec son message, push envoie les commits sur GitHub. Un push avant le commit n'a rien de nouveau à envoyer.